



Departamento de Informática e Matemática Aplicada – DIMAp

gRPC: Introdução

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

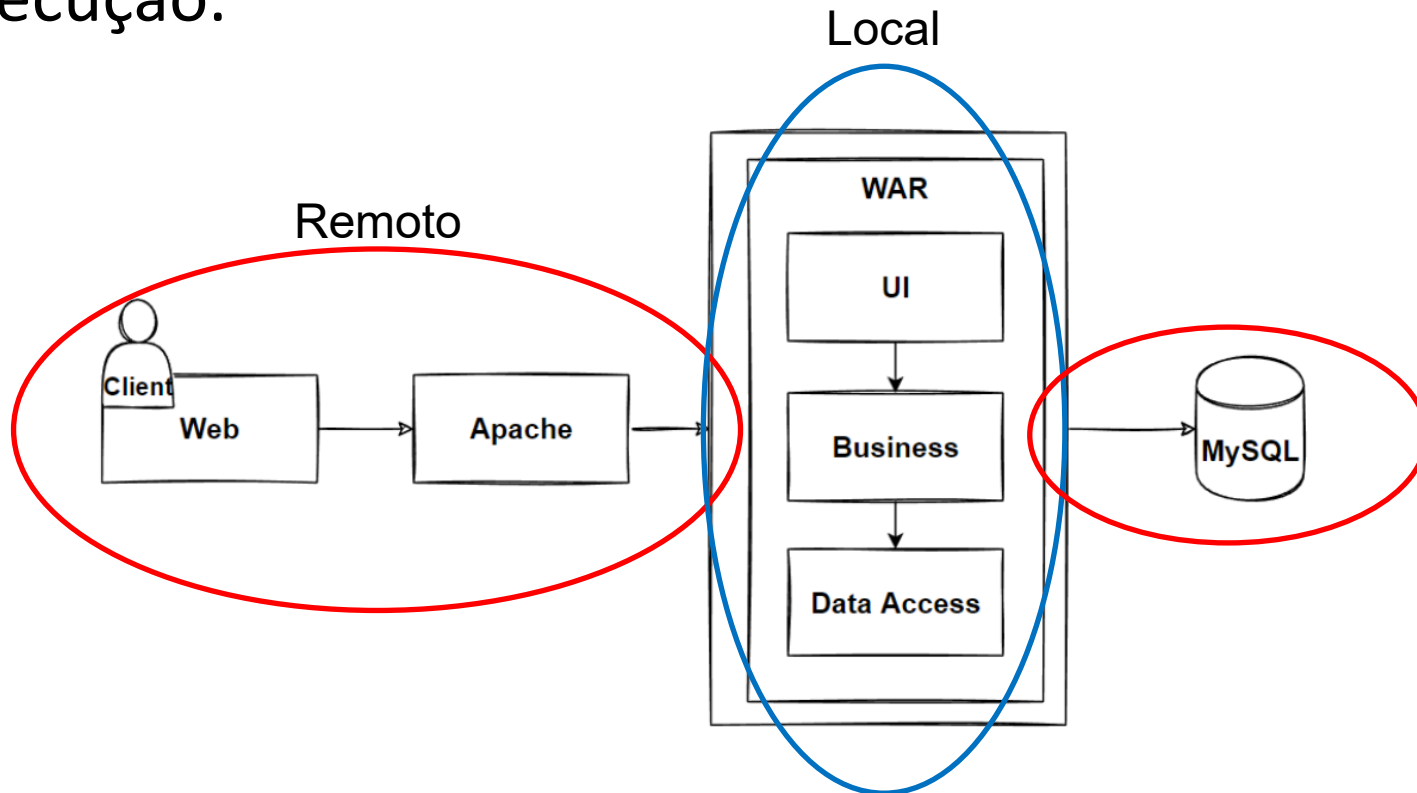
Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.
- Não está autorizada a gravação ou reprodução desta aula

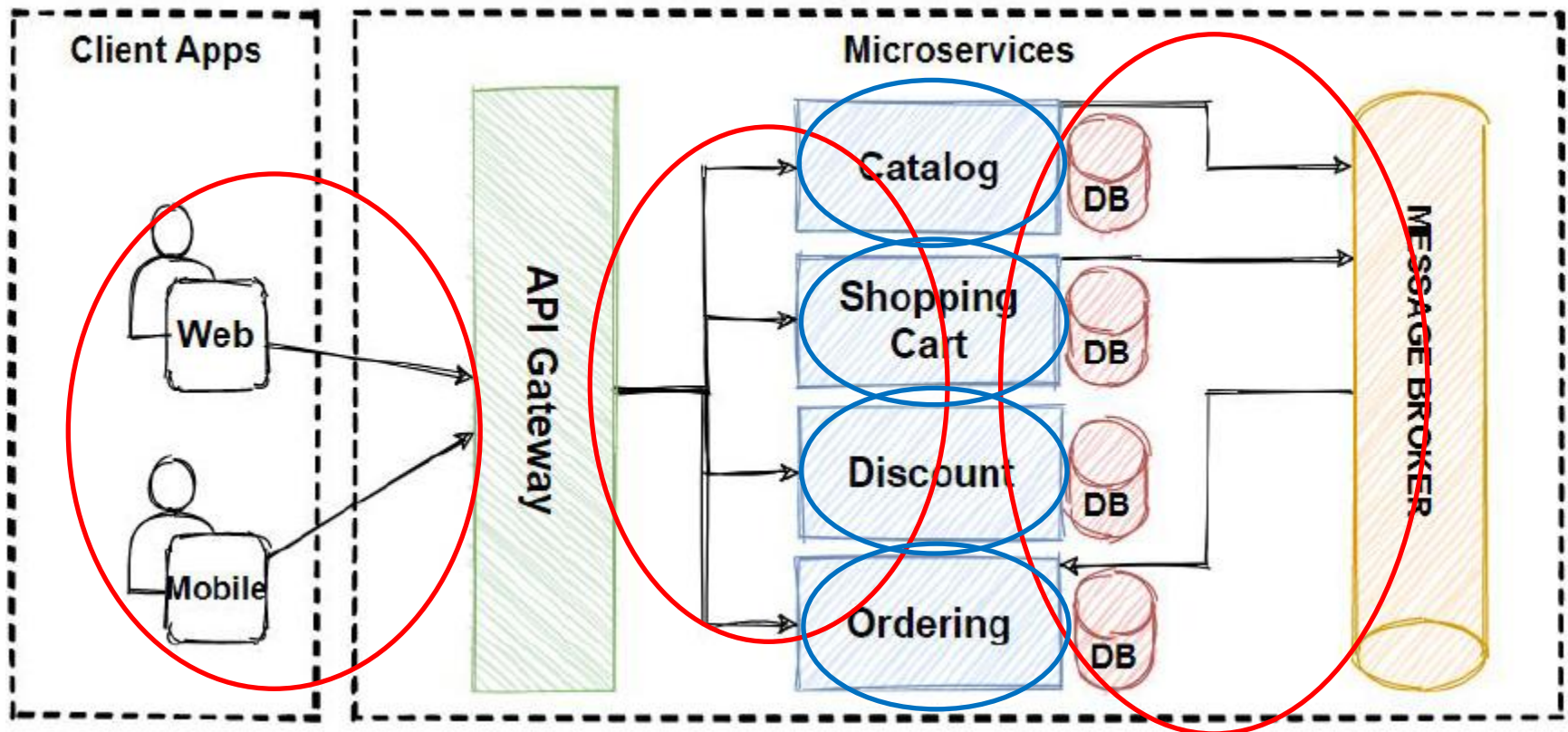
Arquitetura Monolítica

- Numa arquitetura monolítica, as chamadas de métodos são realizadas de forma local, na maior parte da execução.

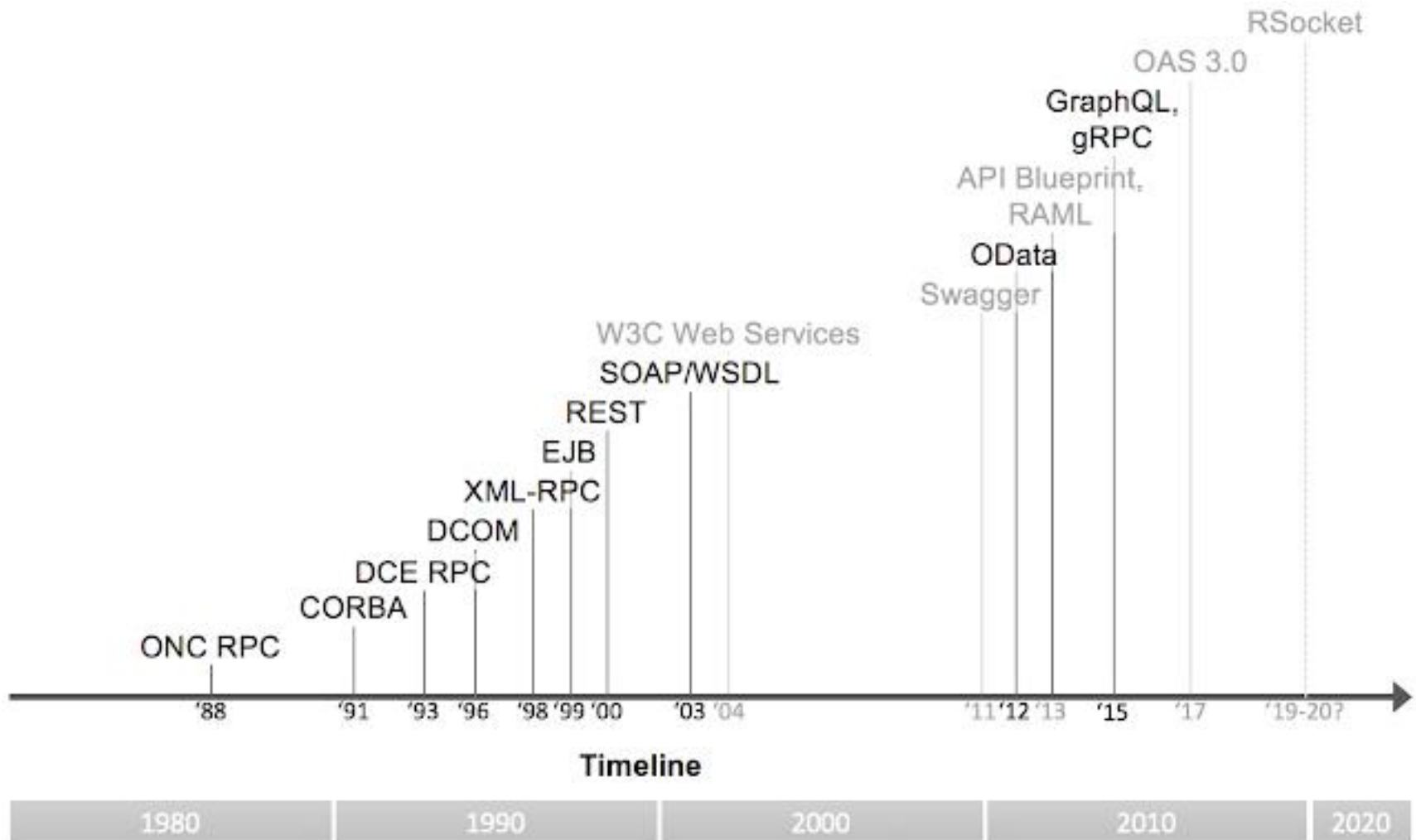


Arquitetura Microserviço

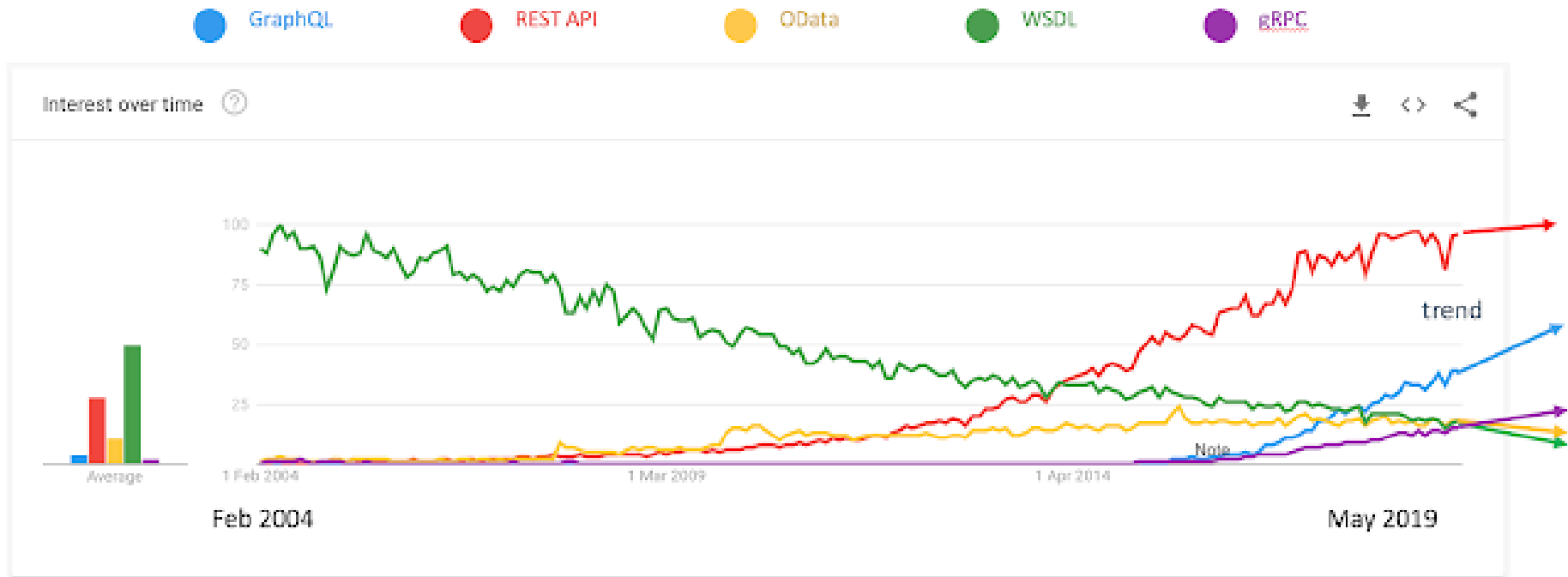
- Com a proliferação da arquitetura microserviço, existe uma grande demanda pela redução da latência nas chamadas remotas



Surgem vários protocolos para tratar deste problema



Surgem vários protocolos para tratar deste problema



Surge o gRPC

- O **gRPC** (gRPC Remote Procedure Call) é um framework de comunicação projetado pelo Google, em 2016, que permite a chamada de métodos remotos entre diferentes sistemas de forma eficiente e escalável.
- As APIs do Google Cloud são acessíveis via gRPC
- Faz parte do Cloud Native Computing Foundation (CNCF)
- Ele é uma evolução do tradicional RPC (Remote Procedure Call) e foi desenvolvido para suportar sistemas distribuídos modernos, como microserviços, onde a comunicação entre os componentes precisa ser rápida e confiável.
- Diversas empresas já adotam o gRPC: Google, Netflix, Microsoft, Uber, Twitter, etc.
- gRPC usa o **Protobuf** como protocolo de comunicação entre as entidades do sistema distribuído.

Diferença de Desempenho

LinkedIn Adopts Protocol Buffers for Microservices Integration and Reduces Latency by up to 60%



LIKE



JUL 19, 2023 • 2 MIN READ

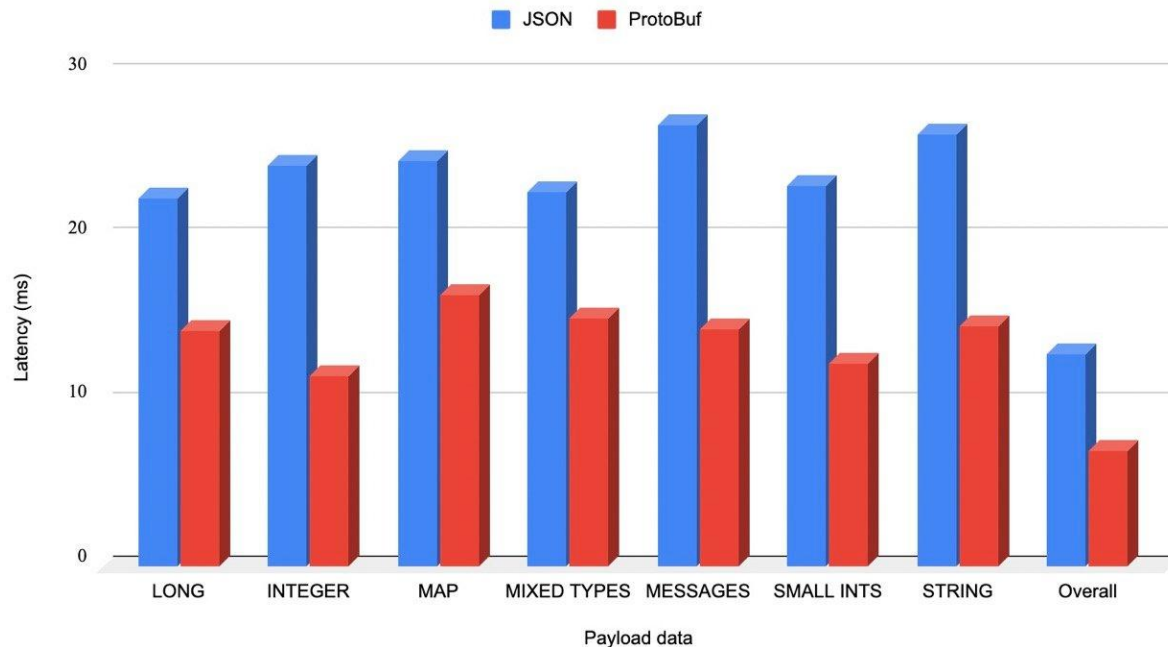
LinkedIn [adopted Protocol Buffers for exchanging data between microservices](#) more efficiently across its platform and integrated it with [Rest.li](#), their open-source REST framework. After the company-wide rollout, they reduced the latency by up to 60% and improved resource utilization at

RELATED CONTENT

QCon San Francisco 2024 Day 1: Architectures,



P99 Latency Comparison



Nossa comparação...

- Resultado do TCC de Mateus Macedo(2024), sob minha orientação, na Residência de Software no TCE/RN comparando o uso de gRPC e JSON/REST na linguagem C#.

Memorando (Id)	Tamanho (MB)	Method	Mean (ms)	Error (%)	StdDev (%)
108830	268,2 MB	restGetById	6014.97	2.83%	8.16%
		grpcGetById	4519.00 (-24.85%)	1.99%	5.61%
108833	19,9 MB	restGetById	565.14	1.98%	4.74%
		grpcGetById	424.48 (-24.90%)	2.00%	4.47%
108834	0,009 MB	restGetById	64.41	3.93%	11.45%
		grpcGetById	34.63 (-46.24%)	1.92%	2.35%

Memorando (Id)	Tamanho (MB)	Method	Gen0	Gen1	Gen2	Allocated (MB)
108830	268,2 MB	restGetById	2	2	2	1853,0
		grpcGetById	2	2	1	544,7 (-70.61%)
108833	19,9 MB	restGetById	1	1	1	227,67
		grpcGetById	-	-	-	40,102(-82.38%)
108834	0,009 MB	restGetById	-	-	-	0,11
		grpcGetById	-	-	-	0,02 (-78.93%)

gRPC no mudo das IAs

- <https://cloud.google.com/blog/products/networking/grpc-as-a-native-transport-for-mcp>

Networking

A gRPC transport for the Model Context Protocol

January 13, 2026

Victor Moreno

Solutions Product Manager

Mark D. Roth

Senior Staff Software Engineer

AI agents are moving from test environments to the core of enterprise operations, where they must interact reliably with external tools and systems to execute complex, multi-step goals. The [Model Context Protocol \(MCP\)](#) is the standard that makes this agent to tool communication possible. In fact, just last month we announced the release of [fully-managed, remote MCP servers](#). Developers can now simply point their AI agents or standard MCP clients like Gemini CLI to a globally-consistent and enterprise-ready endpoint for Google and Google Cloud services.

Por que gRPC é mais rápido que JSON/REST?

- Vamos olhar para o JSON com HTTP:
 - Vantagens:
 - Inteligível para Humanos
 - Browsers podem suportar
 - Contra:
 - Não é type safety
 - Formato em texto (não machine friendly)
 - Tamanho do Payload é relativamente grande
 - Demanda mais bandwidth da rede
 - Leva mais tempo para serializar e deserializar

Por que gRPC é mais rápido que JSON/REST?

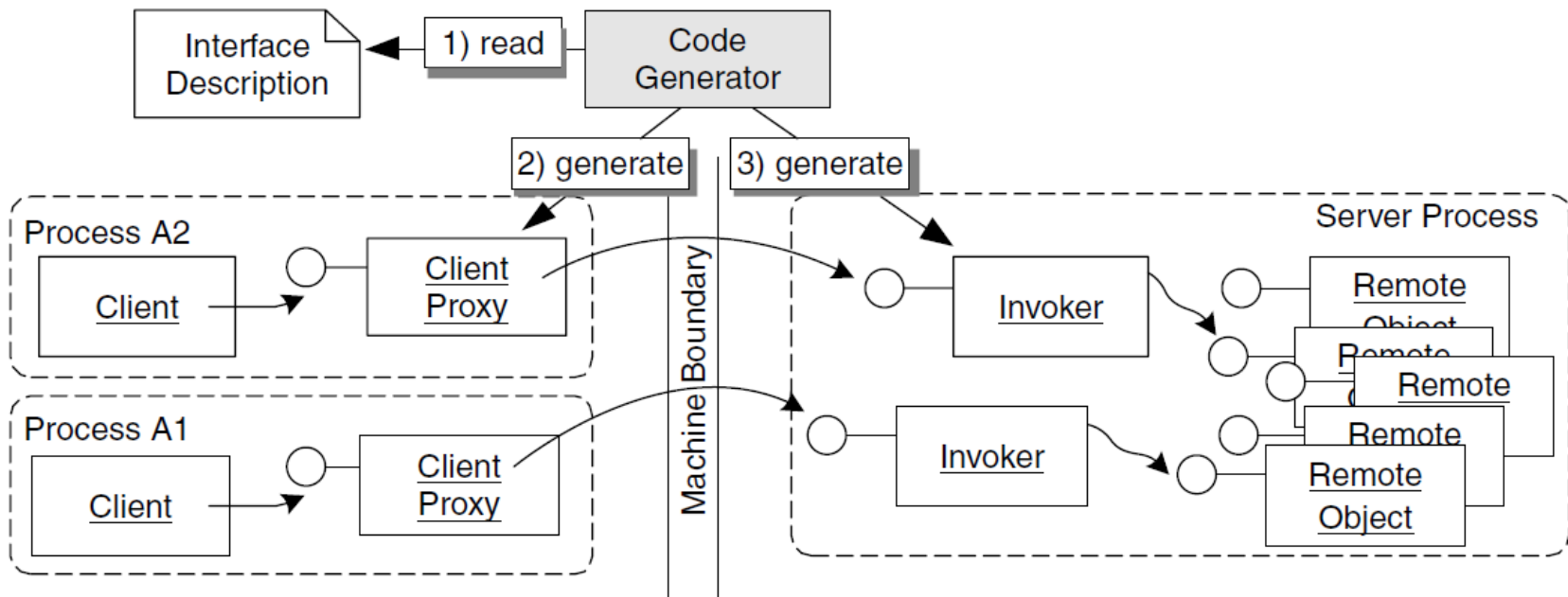
- Vamos olhar para o Protobuf:
 - O gRPC usa o **Protocol Buffers** como sua interface de descrição (IDL) para definir os contratos de serviço e estruturar os dados.
 - O Protobuf é uma ferramenta de serialização eficiente que permite compactar dados em um formato binário leve.
 - O gRPC é otimizado para comunicação de alto desempenho, usando **HTTP/2 ou 3** como protocolo subjacente. Isso permite multiplexação de chamadas, compressão de cabeçalhos e conexões mais rápidas.
 - Oferece suporte a várias linguagens de programação, incluindo Python, Java, Go, C++, Ruby, C#, entre outras, permitindo a interoperabilidade entre sistemas escritos em diferentes tecnologias.

Por que gRPC é mais rápido que JSON/REST?

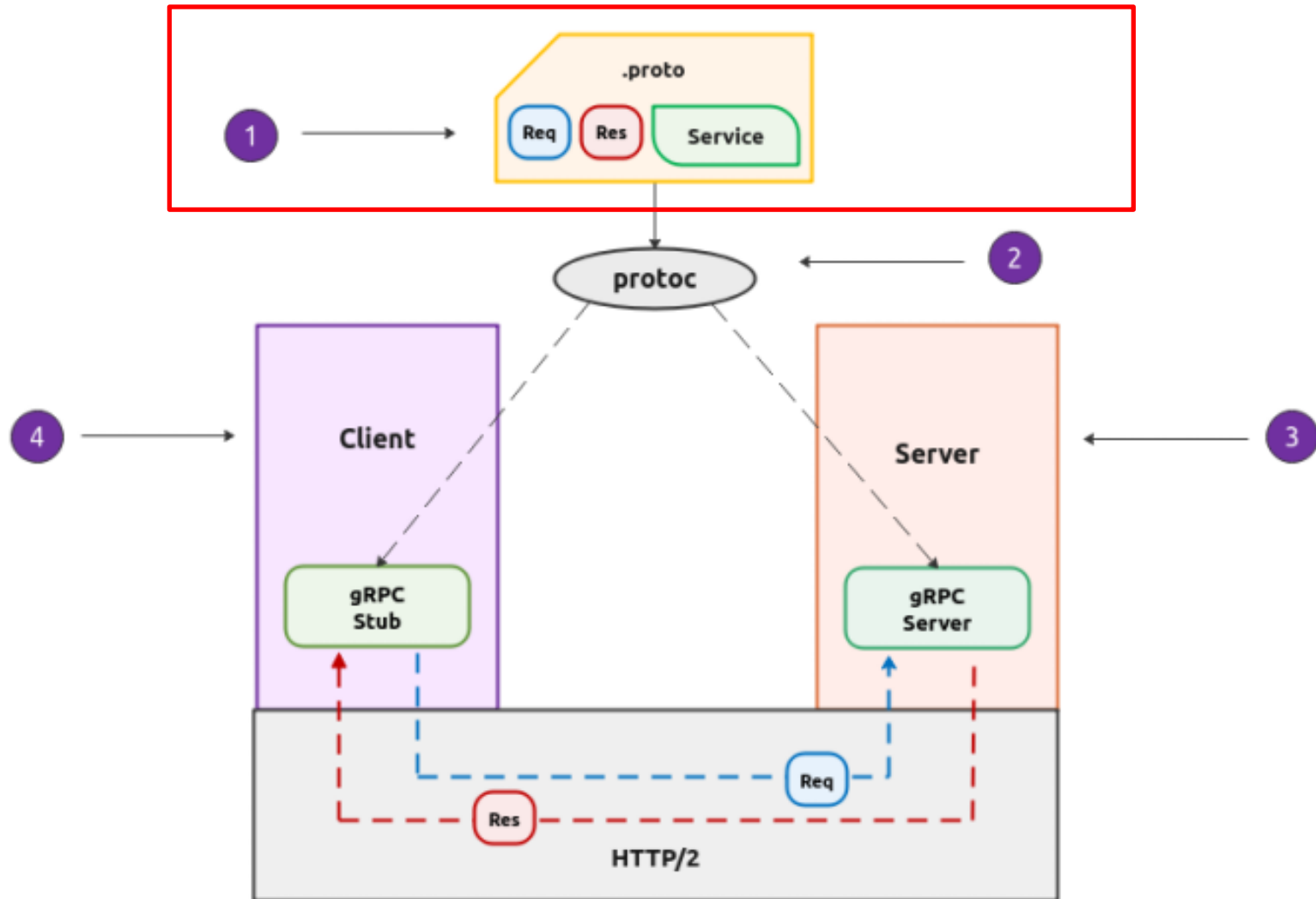
- Vamos olhar para o Protobuf:
 - Desenvolvido pelo Google
 - Plataforma/Linguagem neutra para serialização e deserialização de dados.
 - Usa formato binário:
 - Pode não ser **human friendly**, mas definitivamente é **machine friendly**.
 - Tamanho do payload é relativamente pequeno comparado ao JSON
 - Demanda pouco bandwidth da rede
 - Demanda menos tempo para serializar e deserializar

Protobuf na arquitetura de Middleware

- O protobuf é como uma interface disponibilizada pelo serviço remoto e contempla a descrição das mensagens e dos serviços



Vamos definir nossa interface por meio do arquivo .proto



Protocol Buffers

- Uma descrição Protocol Buffers é composta pelas mensagens e pelos serviços.

```
syntax = "proto3";  
package projetogrpc;  
option java_package = "br.imd.ufrn";  
option java_multiple_files = true;  
  
message Cliente {  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    sint32 saldo = 7;  
}
```


Protocol Buffers

- `syntax = "proto3";`
 - Define a versão da sintaxe do Protocol Buffers que está sendo usada.
- `package projetogrpc;`
 - Define um namespace ou pacote para os serviços e mensagens descritos no arquivo `.proto`
- `option java_multiple_files = true;`
 - Indica que cada mensagem, serviço ou enum descrito no arquivo `.proto` deve ser gerado em arquivos Java separados, em vez de tudo ser gerado em um único arquivo.
- `option java_package = "br.imd.ufrn";`
 - Define o **pacote Java** em que o código gerado será colocado.

Anatomia de uma Mensagem Protobuf

- A Estrutura Básica:
 - Uma mensagem é definida pela palavra-chave ***message***.
 - Contém **campos**, que são os dados da mensagem.
- Componentes de um **campo**: Cada campo em uma mensagem tem três partes cruciais:
 - **Tipo de Dado**: O tipo da informação (ex: string, int32, bool, repeated).
 - **Nome do Campo**: Um nome descritivo para acessar o dado (ex: user_id, email).
 - **Número do Campo**: Um identificador único para serialização (ex: 1, 2, 3).

Exemplo de *message*

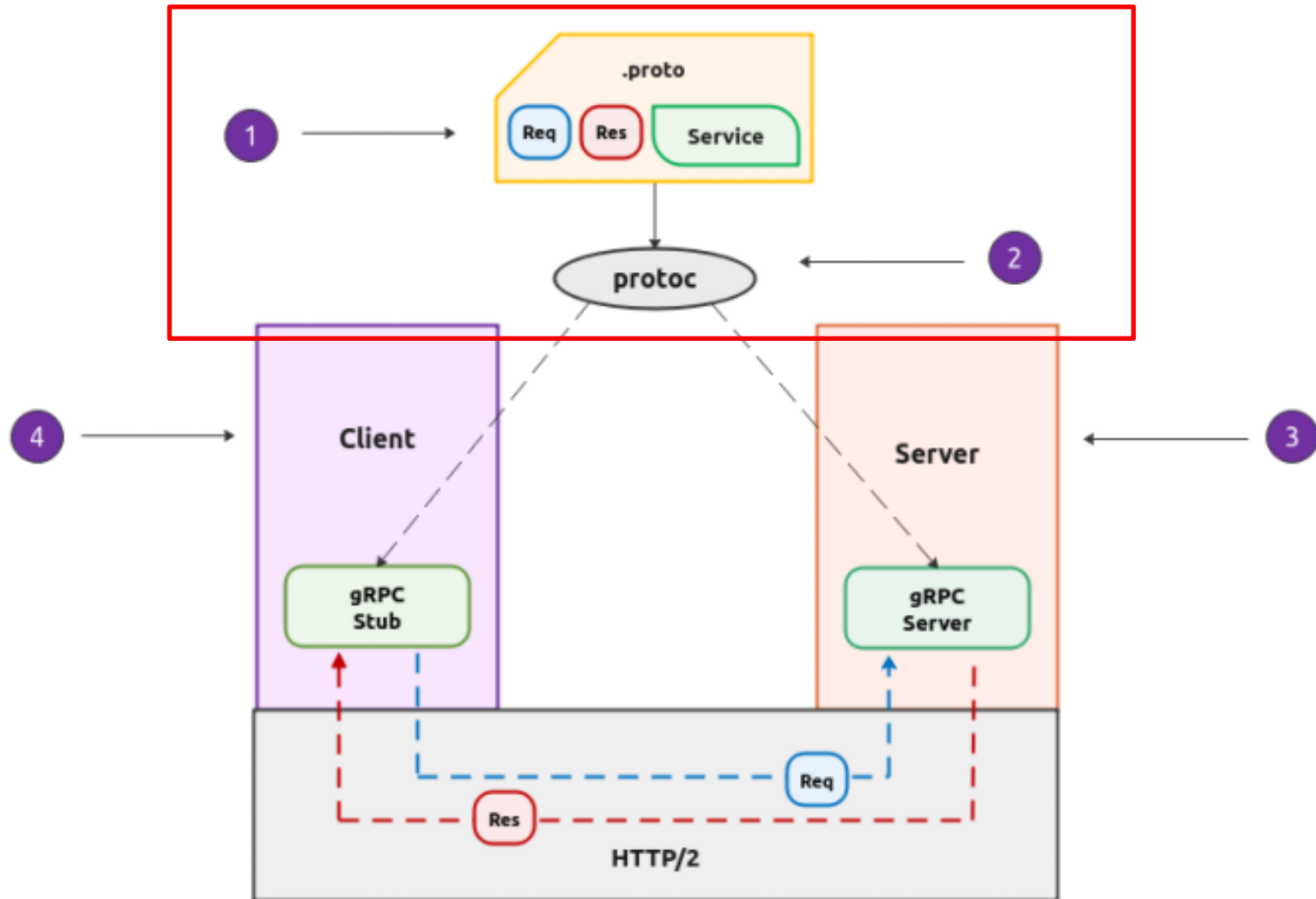
```
message Cliente {  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    sint32 saldo = 7;  
}
```

Mapeamento entre tipos de Java e Proto buffer.

Java Type	Proto Type
int	int32 / sint32
long	int64 / sint64
float	float
double	double
boolean	bool
String	string
byte[]	bytes

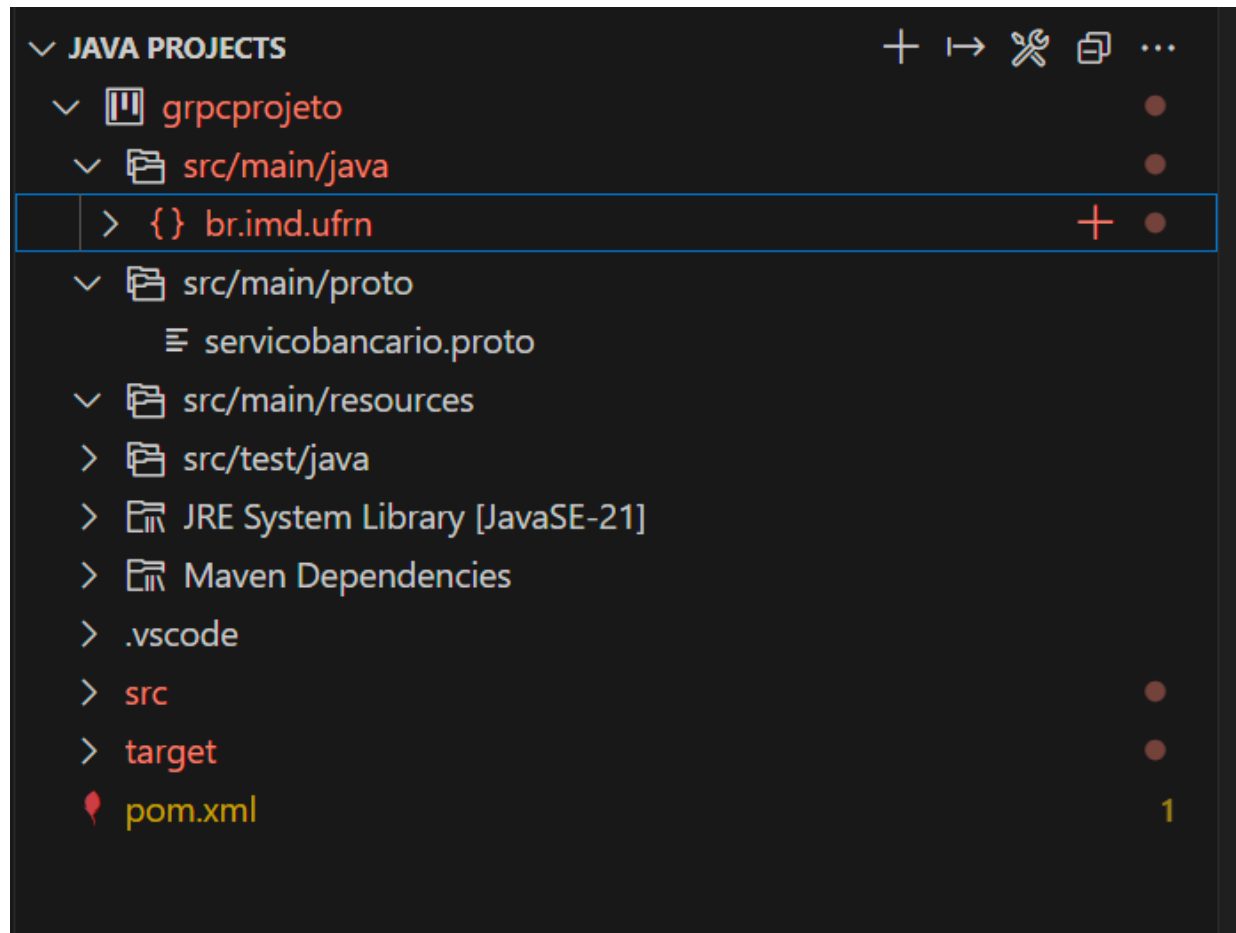
```
message Cliente {  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    sint32 saldo = 7;  
}
```

Vamos gerar os Stubs passando o arquivo .proto



Projeto Maven para acomodar seu projeto gRPC

- Devemos criar um projeto Maven onde temos a pasta `src/main/proto`



Deve-se importar as seguintes dependência:

```
<dependency>
  <groupId>io.grpc</groupId>
  <artifactId>grpc-netty-shaded</artifactId>
  <version>${grpc.version}</version>
</dependency>
<dependency>
  <groupId>io.grpc</groupId>
  <artifactId>grpc-protobuf</artifactId>
  <version>${grpc.version}</version>
</dependency>
<dependency>
  <groupId>io.grpc</groupId>
  <artifactId>grpc-stub</artifactId>
  <version>${grpc.version}</version>
</dependency>
```

Compilar usando o Maven

- Na sequência, é preciso compilar o projeto Maven para que o arquivo .proto seja interpretado e gerado os arquivos **gRPC stub** e **gRPC server**.

```
PS X:\> mvn compile
[INFO] Scanning for projects...
[WARNING]
[WARNING] Some problems were encountered while building the effective model for br.imd.ufrn:grpcprojeto:jar:1.0-SNAPSHOT
[WARNING] 'build.plugins.plugin.version' for org.apache.maven.plugins:maven-compiler-plugin is missing. @ line 84, column 21
[INFO] --- resources:3.3.1:resources (default-resources) @ grpcprojeto ---
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO] Copying 1 resource from src\main\proto to target\classes
[INFO] Copying 1 resource from src\main\proto to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ grpcprojeto ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 7 source files with javac [debug target 21] to target\classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```


Dica

- Se o seu path tiver espaço em branco, sugiro criar um link:
 - `subst X: "C:\Users\nelio\OneDrive\Disciplinas\Aulas Gerais de Programação Distribuída\gRPC\Introducao\grpcprojeto"`
 - `cd X:`
 - `mvn clean`
 - `mvn compile`
 - `mvn install`

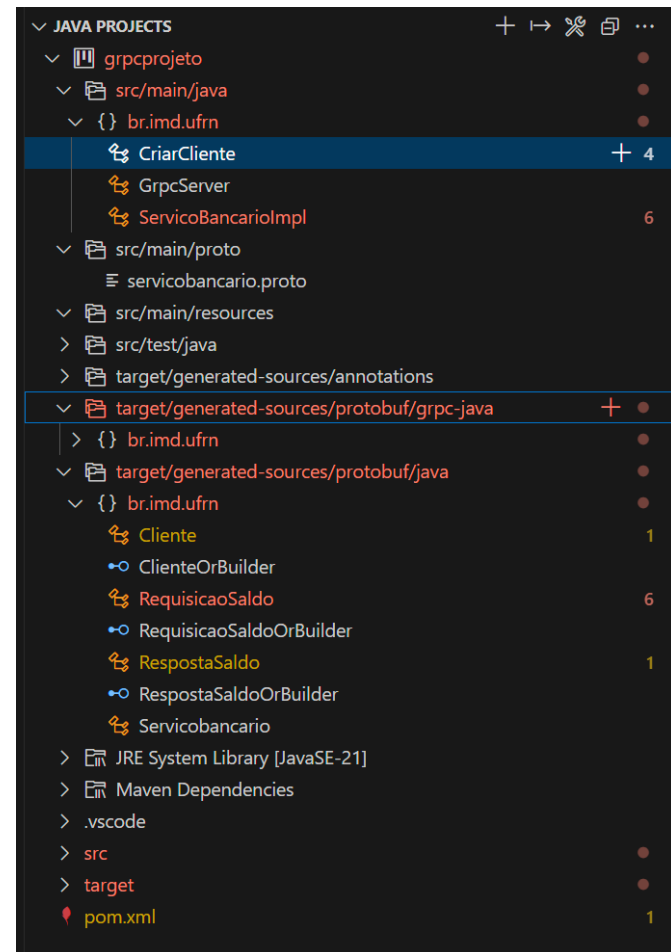
O que acontece se repetir o identificador

```
message Cliente {  
    string nome = 1;  
    int32 idade = 1;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;
```

```
[ERROR] Failed to execute goal io.github.ascpes:protobuf-maven-plugin:3.6.1:generate (default) on project grpcprojeto: Generation failed - PROTOC FAILED: Protoc failed with an error. Check the build logs above to find the root cause. -> [Help 1]  
[ERROR]  
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.  
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
```

Se tudo der certo, deve-se ter uma pasta target...

- Ao final da compilação, é gerado um conjunto de arquivos nas pastas ***target***.



Exemplo de classe criada

- Note que a **message** Cliente foi convertida numa classe com dezenas de métodos:

```
/**
 * <code>double salario = 5;</code>
 * @param value The salario to set.
 * @return This builder for chaining.
 */
public Builder setSalario(double value) {

    salario_ = value;
    bitField0_ |= 0x00000010;
    onChanged();
    return this;
}
/**
```

```
// Protobuf Java Version: 3.25.5
package br.imd.ufrrn;

/**
 * Protobuf type {@code projetogrpc.Cliente}
 */
public final class Cliente extends
    com.google.protobuf.GeneratedMessageV3 implements
    // @@protoc_insertion_point(message_implementation:projetogrpc.Cliente)
    ClienteOrBuilder {
private static final long serialVersionUID = 0L;
    // Use Cliente.newBuilder() to construct.
    private Cliente(com.google.protobuf.GeneratedMessageV3.Builder<?> builder) {
        super(builder);
    }
    private Cliente() {
        nome_ = "";
        email_ = "";
    }

    @java.lang.Override
    @SuppressWarnings({"unused"})
    protected java.lang.Object newInstance(
        UnusedPrivateParameter unused) {
        return new Cliente();
    }

    public static final com.google.protobuf.Descriptors.Descriptor
        getDescriptor() {
        return br.imd.ufrrn.Teste.internal_static_projetogrpc_Cliente_descriptor;
    }

    @java.lang.Override
    protected com.google.protobuf.GeneratedMessageV3.FieldAccessorTable
        internalGetFieldAccessorTable() {
        return br.imd.ufrrn.Teste.internal_static_projetogrpc_Cliente_fieldAccessorTabl
            .ensureFieldAccessorsInitialized(
                messageClass:br.imd.ufrrn.Cliente.class, builderClass:br.imd.ufrrn.Clier
            );
    }
}
```

Exemplo de classe criada

```
@java.lang.Override
public void writeTo(com.google.protobuf.CodedOutputStream output)
    throws java.io.IOException {
    if (!com.google.protobuf.GeneratedMessage.isStringEmpty(nome_)) {
        com.google.protobuf.GeneratedMessage.writeString(output, fieldNumber: 1, nome_);
    }
    if (idade_ != 0) {
        output.writeInt32(fieldNumber: 2, idade_);
    }
    if (!com.google.protobuf.GeneratedMessage.isStringEmpty(email_)) {
        com.google.protobuf.GeneratedMessage.writeString(output, fieldNumber: 3, email_);
    }
    if (vinculoempreg_ != false) {
        output.writeBool(fieldNumber: 4, vinculoempreg_);
    }
    if (java.lang.Double.doubleToRawLongBits(salario_) != 0) {
        output.writeDouble(fieldNumber: 5, salario_);
    }
    if (numeroConta_ != 0) {
        output.writeInt32(fieldNumber: 6, numeroConta_);
    }
}
```

```
message Cliente {

    string nome = 1;
    int32 idade = 2;
    string email = 3;
    bool vinculoempreg = 4;
    double salario = 5;
    int32 numero_conta = 6;
    sint32 saldo = 7;

}
```

Como criar uma instância de Cliente ?

```
public static void main(String[] args) {  
  
    var cliente = Cliente.newBuilder()  
        .setNome(value:"Nelio Cacho")  
        .setIdade(value:30)  
        .setEmail(value:"neliocacho@gmail.com")  
        .setVinculoempreg(value:true)  
        .setSalario(value:1000.00)  
        .setNumeroConta(value:123456)  
        .setSaldo(value:100.00)  
        .build();  
  
    log.info(format:"{}", cliente);  
  
}
```

```
message Cliente {  
  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    sint32 saldo = 7;  
  
}
```

```
14:32:24.217 [main] INFO br.imd.ufrn.CriarCliente -- nome: "Nelio Cacho"  
idade: 30  
email: "neliocacho@gmail.com"  
vinculoempreg: true  
salario: 1000.0  
numero_conta: 123456  
saldo: 100.0
```

Composição de Message

- Pode-se criar **message** com estruturas de dados mais complexas por meio da composição de **message**.

```
message Endereco {  
    string rua = 1;  
    string bairro = 2;  
    string cidade = 3;  
    string estado = 4;  
    string cep = 5;  
}
```

```
message Cliente {  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    double saldo = 7;  
    Endereco endereco = 8;  
}
```

Composição de Message

```
var endereco = Endereco.newBuilder()  
    .setRua("Rua Prudente de Moraes")  
    .setBairro("Centro")  
    .setCidade("Natal")  
    .setEstado("RN")  
    .setCep("59150-678")  
    .build();  
var cliente = Cliente.newBuilder()  
    .setNome("Nelio Cacho")  
    .setIdade(30)  
    .setEmail("neliocacho@gmail.com")  
    .setVinculoempreg(true)  
    .setSalario(1000.00)  
    .setNumeroConta(123456)  
    .setSaldo(100.00)  
    .setEndereco(endereco)  
    .build();  
log.info(format:"{}", cliente);
```


Composição de Message

```
14:34:53.509 [main] INFO br.imd.ufrn.CriarCliente -- nome: "Nelio Cacho"
idade: 30
email: "neliocacho@gmail.com"
vinculoempreg: true
salario: 1000.0
numero_conta: 123456
saldo: 100.0
endereco {
  rua: "Rua Prudente de Moraes"
  bairro: "Centro"
  cidade: "Natal"
  estado: "RN"
  cep: "59150-678"
}
```

Listas e Coleções

- Use **repeated** para definir um campo que pode ter zero ou mais ocorrências.
- Funciona como um array, lista ou conjunto(set) na linguagem gerada.

```
message Cliente {  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    double saldo = 7;  
    Endereco endereco = 8;  
    repeated Dependente dependentes = 9;  
}  
  
message Dependente {  
    string nome = 1;  
    int32 idade = 2;  
    string parentesco = 3;  
}
```

Listas e Coleções

```
var dependente1 = Dependente.newBuilder()
    .setNome("Maria Cacho")
    .setIdade(5)
    .setParentesco("Filha")
    .build();
var dependente2 = Dependente.newBuilder()
    .setNome("João Cacho")
    .setIdade(3)
    .setParentesco("Filho")
    .build();
var cliente = Cliente.newBuilder()
    .setNome("Nelio Cacho")
    .setIdade(30)
    .setEmail("neliocacho@gmail.com")
    .setVinculoempreg(true)
    .setSalario(1000.00)
    .setNumeroConta(123456)
    .setSaldo(100.00)
    .setEndereco(endereco)
    // .addDependentes(dependente1)
    // .addDependentes(dependente2)
    // .addAllDependentes(Set.of(dependente1, dependente2))
    .addAllDependentes(List.of(dependente1, dependente2))
    .build();
```

Caso queira
usar um Set
em Java



Listas e Coleções

```
15:15:43.376 [main] INFO br.imd.ufrn.CriarCliente -- nome: "Nelio Cacho"
idade: 30
email: "neliocacho@gmail.com"
vinculoempreg: true
salario: 1000.0
numero_conta: 123456
saldo: 100.0
endereco {
  rua: "Rua Prudente de Moraes"
  bairro: "Centro"
  cidade: "Natal"
  estado: "RN"
  cep: "59150-678"
}
dependentes {
  nome: "Maria Cacho"
  idade: 5
  parentesco: "Filha"
}
dependentes {
  nome: "João Cacho"
  idade: 3
  parentesco: "Filho"
}
```

Definindo Maps

- Permite definir coleções de chave-valor (como um Map ou Dictionary).
- A chave deve ser um tipo escalar.
- O valor pode ser qualquer tipo (escalar ou mensagem).

```
message ChavePix {  
    string chave = 1;  
}  
message Cliente {  
    string nome = 1;  
    int32 idade = 2;  
    string email = 3;  
    bool vinculoempreg = 4;  
    double salario = 5;  
    int32 numero_conta = 6;  
    double saldo = 7;  
    Endereco endereco = 8;  
    repeated Dependente dependentes = 9;  
    map<string, ChavePix> chaves_pix = 10;  
}
```

Definindo Maps

```
public class CriarClienteMap {  
  
    private static final Logger log = LoggerFactory.getLogger(clazz: CriarCliente.class);  
  
    Run main | Debug main | Run | Debug  
    public static void main(String[] args) {  
  
        var chavaPix1 = ChavePix.newBuilder()  
            .setChave(value: "teste@gmail.com")  
            .build();  
  
        var cliente = Cliente.newBuilder()  
            .setNome(value: "Nelio Cacho")  
            .setIdade(value: 30)  
            .setEmail(value: "neliocacho@gmail.com")  
            .setVinculoempreg(value: true)  
            .setSalario(value: 1000.00)  
            .setNumeroConta(value: 123456)  
            .setSaldo(value: 100.00)  
            .putChavesPix(key: "nelio", chavaPix1)  
            .build();  
        log.info(format: "{}", cliente);  
    }  
}
```

Definindo Maps

```
SLF4J(I): Connected with provider of type [ch.qos.logback.classic.spi.LogbackServiceProvider]
15:56:55.748 [main] INFO br.imd.ufrn.CriarCliente -- nome: "Nelio Cacho"
idade: 30
email: "neliocacho@gmail.com"
vinculoempreg: true
salario: 1000.0
numero_conta: 123456
saldo: 100.0
chaves_pix {
    key: "nelio"
    value {
        chave: "teste@gmail.com"
    }
}
```

Definindo Enums

- Definem um conjunto de valores nomeados, limitando as opções para um campo.
- Garante a validade e clareza dos dados.
- O primeiro valor deve ter o número 0

```
enum TipoChavePix {  
    CPF = 0;  
    CNPJ = 1;  
    EMAIL = 2;  
    TELEFONE = 3;  
    ALEATORIA = 4;  
}  
  
message ChavePix {  
    string chave = 1;  
    TipoChavePix tipo = 2;  
}
```


Definindo Enums

```
var chavaPix1 = ChavePix.newBuilder()  
    .setChave(value:"teste@gmail.com")  
    .setTipo(TipoChavePix.EMAIL)  
    .build();  
var cliente = Cliente.newBuilder()  
    .setNome(value:"Nelio Cacho")  
    .setIdade(value:30)  
    .setEmail(value:"neliocacho@gmail.com")  
    .setVinculoempreg(value:true)  
    .setSalario(value:1000.00)  
    .setNumeroConta(value:123456)  
    .setSaldo(value:100.00)  
    .putChavesPix(key:"nelio", chavaPix1)  
    .build();  
log.info(format:"{}", cliente);
```

Definindo Enums

```
17:22:25.046 [main] INFO br.imd.ufrn.CriarClienteMap -- nome: "Nelio Cacho"
idade: 30
email: "neliocacho@gmail.com"
vinculoempreg: true
salario: 1000.0
numero_conta: 123456
saldo: 100.0
chaves_pix {
  key: "nelio"
  value {
    chave: "teste@gmail.com"
    tipo: EMAIL
  }
}
```

Comparando o desempenho de gRPC vs JSON

- Imagine esse cenário onde tenho o mesmo objeto definido em Java/gRPC e em Java/Json.

```
// Criando o cliente com Protobuf
var protoEndereco = Endereco.newBuilder()
    .setRua(value:"Rua Prudente de Moraes")
    .setBairro(value:"Centro")
    .setCidade(value:"Natal")
    .setEstado(value:"RN")
    .setCep(value:"59150-678")
    .build();

var protoCliente = Cliente.newBuilder()
    .setNome(value:"Nelio Cacho")
    .setIdade(value:30)
    .setEmail(value:"neliocacho@gmail.com")
    .setVinculoempreg(value:true)
    .setSalario(value:1000.00)
    .setNumeroConta(value:123456)
    .setSaldo(value:100.00)
    .setEndereco(protoEndereco)
    .build();
```

Comparando o desempenho

- Imagine esse cenário onde tenho o mesmo objeto definido em Java/gRPC e em Java/Json.

```
var jsonendereco = new JsonEndereco(  
    rua:"Rua Prudente de Moraes",  
    bairro:"Centro",  
    cidade:"Natal",  
    estado:"RN",  
    cep:"59150-678"  
);  
var jsonCliente = new JsonCliente(  
    nome:"Nelio Cacho",  
    idade:30,  
    email:"neliocacho@gmail.com",  
    vinculoempreg:true,  
    salario:1000.00,  
    numero_conta:123456,  
    saldo:100.00,  
    jsonendereco  
);
```

Vamos Transformar ambos em bytes, já que será o payload quando for ser enviado pela Rede

```
private static void proto(Cliente cliente){
    try {
        var bytes = cliente.toByteArray();
        log.info(format:"tamanho do Cliente em Proto: {}", bytes.length);
        Cliente.parseFrom(bytes);
    } catch (InvalidProtocolBufferException e) {
        throw new RuntimeException(e);
    }
}

private static void json(JsonCliente cliente){
    try{
        var bytes = mapper.writeValueAsBytes(cliente);
        log.info(format:"tamanho do Cliente em json: {}", bytes.length);
        mapper.readValue(bytes, valueType:JsonCliente.class);
    }catch (Exception e){
        throw new RuntimeException(e);
    }
}
```

Comparando o desempenho de gRPC vs JSON

- Ao executar o programa, nota-se que a quantidade de bytes para enviar um objeto Json é mais que o dobro do objeto em gRPC.

```
INFO br.imd.ufrn.TesteDesempenho -- tamanho do Cliente em json: 249  
INFO br.imd.ufrn.TesteDesempenho -- tamanho do Cliente em Proto: 117
```

Quanto isso representa em termos de tempo de processamento ?

- A forma correta seria usar o JMH para realizar tal comparação, mas para simplificar vamos executar isso milhões de vezes....

```
private static void runTest(String abordagem, Runnable runnable){  
    var start = System.currentTimeMillis();  
    for (int i = 0; i < 5_000_000; i++) {  
        runnable.run();  
    }  
    var end = System.currentTimeMillis();  
    log.info(format:"Tempo para concluir o teste: {} - {} ms", abordagem, (end - start));  
}
```

Quanto isso representa em termos de tempo de processamento ?

- Quando chamo o código abaixo, temos o seguintes resultado:

```
for (int i = 0; i < 5; i++) {  
    runTest(abordagem:"json", () -> json(jsonCliente));  
    runTest(abordagem:"proto", () -> proto(protoCliente));  
}
```

```
Tempo para concluir o teste: json - 6087 ms  
Tempo para concluir o teste: proto - 1628 ms  
Tempo para concluir o teste: json - 5807 ms  
Tempo para concluir o teste: proto - 1492 ms  
Tempo para concluir o teste: json - 5761 ms  
Tempo para concluir o teste: proto - 1506 ms  
Tempo para concluir o teste: json - 5785 ms  
Tempo para concluir o teste: proto - 1484 ms  
Tempo para concluir o teste: json - 5799 ms  
Tempo para concluir o teste: proto - 1490 ms
```

Protobuf é muito *mais rápido* que JSON nesse teste de desempenho.

Se eu usar poucos campos da minha classe, o que acontece?

- Ao executar, temos uma diferença enorme entre os tamanhos de payload gerados.

```
tamanho do Cliente em json: 177  
tamanho do Cliente em Proto: 8
```

```
// Criando o cliente com Protobuf  
var protoEndereco = Endereco.newBuilder()  
    .setEstado(value:"RN")  
    .build();  
var protoCliente = Cliente.newBuilder()  
    .setVinculoempreg(value:true)  
    .setEndereco(protoEndereco)  
    .build();  
// criando o cliente com JSON  
var jsonendereco = new JsonEndereco();  
jsonendereco.setBairro(bairro:"RN");  
var jsonCliente = new JsonCliente();  
jsonCliente.setVinculoempreg(vinculoempreg:  
jsonCliente.setEndereco(jsonendereco);  
// Executando os testes de desempenho  
json(jsonCliente);  
proto(protoCliente);
```

Como funciona o Protocol Buffers?

- Suponha esse exemplo:

Proto definition

```
{  
  string name = 1;  
  int32 age = 2;  
}
```

```
{  
  score : 25,  
  name: "Tom"  
}
```

Representação em JSON
ocuparia 26 bytes

```
08 C8 01 12 03 54 6F 6D
```

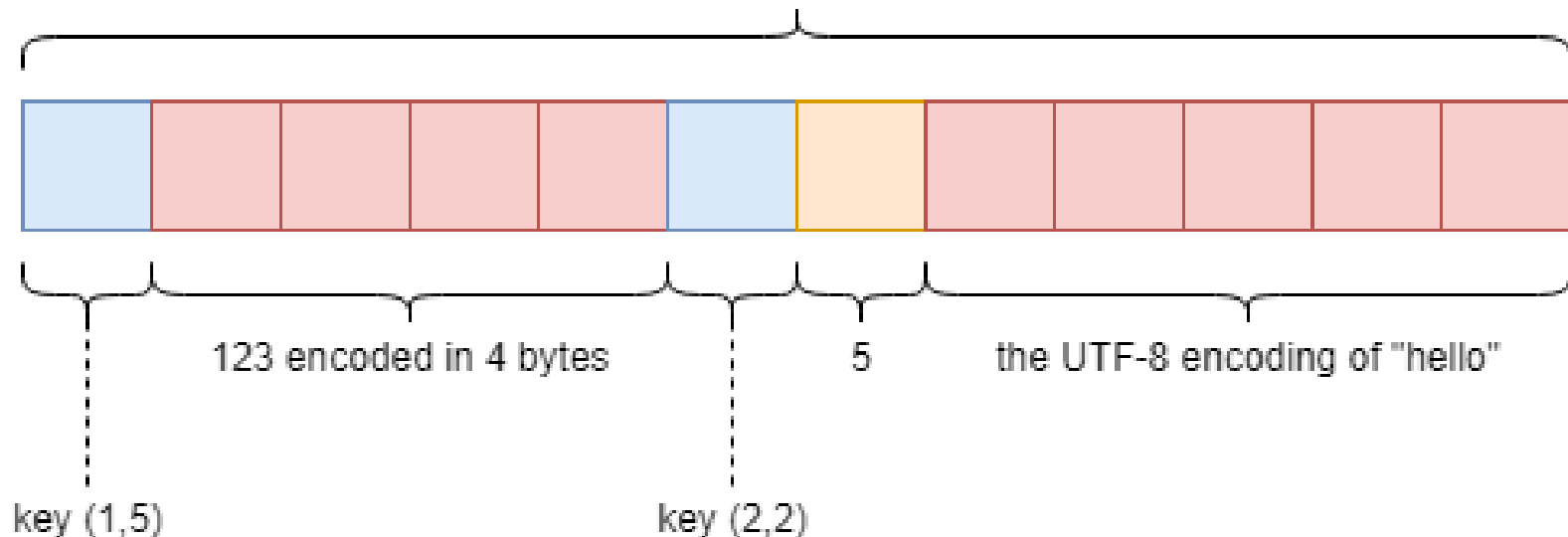
Representação em Proto
Buffer ocupa 8 bytes

Protocol Buffers

```
message Thing {  
  fixed32 a = 1;  
  string b = 2;  
}
```

```
var thing1 = Thing(a: 123, b: "hello")
```

A byte blob containing the proto encoding of `thing1`



Na próxima aula vamos aprender como definir Serviços Remotos em gRPC

